

Getting a union of two arrays in JavaScript [duplicate]

Asked 13 years ago Modified 6 months ago Viewed 205k times



96



This question already has answers here:

[How to merge two arrays in JavaScript and de-duplicate items](#) (90 answers)

Closed 5 years ago.

Say I have an array of `[34, 35, 45, 48, 49]` and another array of `[48, 55]`. How can I get a resulting array of `[34, 35, 45, 48, 49, 55]`?

javascript

arrays

Share Improve this question Follow

edited Oct 24, 2012 at 15:26



crdx

1,422 ● 13 ● 26

asked Sep 2, 2010 at 17:58



CFNinja

3,571 ● 4 ● 38 ● 59

with ES6 goodies it will be just a oneliner with no dependencies. Sad that we need to wait a lot of time before it will be available in all modern browsers. Anyway, check [my answer](#) – [Salvador Dali](#) Jan 17, 2015 at 7:06

[This](#) implementation in Code Golf is IMO the most elegant and efficient solution out there. – [Sayan](#) Oct 19, 2015 at 6:16

21 Answers

Sorted by:

Highest score (default)



189



With the arrival of ES6 with sets and spread operator, you can write the following cryptic one liner:

```
var a = [34, 35, 45, 48, 49];  
var b = [48, 55];  
var union = [...new Set([...a, ...b])];  
console.log(union);
```

Run code snippet

Expand snippet

Little explanation about this line: `[...a, ...b]` concatenates two arrays, you can use `a.concat(b)` as well. `new Set()` create a set out of it and thus your union. And the last `[...x]` converts it back to an array.

Share Improve this answer Follow

edited Mar 23 at 17:03

 Shea Hunter Belsky
Hunte 2,825 ● 3 ● 22 ● 31

answered Jan 17, 2015 at 7:03

 Salvador Dali
214k ● 147 ● 704 ● 753

what about performance – RPallas Mar 18, 2015 at 12:10

- 1 @RPallas I have not done any testing (you can easily do them here jsperf.com). But all the functions here are native, which suggests that it should be faster then anything non-native. – Salvador Dali Mar 18, 2015 at 19:50

Performance test here: jsperf.com/union-speeds/1 It's a bit slower than the lodash solution, but unlikely to matter unless you're performing lots of big unions in real-time. And if that's the case, you should probably look into an even faster solution like Lazy.js – Tony Boyles May 30, 2017 at 17:44

- 4 Typescript was complaining and I replaced the spread operator with `Array.from(new Set([...a, ...b]))` and it worked. – Agu V Apr 3, 2019 at 13:36

If you don't need to keep the order, and consider `45` and `"45"` to be the same:

```
function union_arrays (x, y) {
  var obj = {};
  for (var i = x.length-1; i >= 0; -- i)
    obj[x[i]] = x[i];
  for (var i = y.length-1; i >= 0; -- i)
    obj[y[i]] = y[i];
  var res = []
  for (var k in obj) {
    if (obj.hasOwnProperty(k)) // <-- optional
      res.push(obj[k]);
  }
  return res;
}

console.log(union_arrays([34,35,45,48,49], [44,55]));
```

🔍 Run code snippet

🔗 Expand snippet

Share Improve this answer Follow

edited Feb 15, 2019 at 11:43

 Tushar Walzade
3,737 ● 4 ● 33 ● 56

answered Sep 2, 2010 at 18:05

 kennytym
512k ● 107 ● 1086 ● 1005

don't care about the order. this worked fine. thx – CFNinja Sep 2, 2010 at 19:26

- 2 This will fail for arrays of objects. `union_arrays([a:1], [b:2])` will return `[b:2]`. – EricP Jul 4, 2014 at 14:39

Why not skip declaring `res` and the whole "for(var k in obj)... push..." and just return `Object.keys(obj)`? You can also combine those two initial for-loops into one with "var both = x.concat(y)" to concatenate the two arrays into one. That way, you can union any number of arrays, too. – Jemenake May 27, 2015 at 4:12

@Jemenake `Object.keys()` does not exist in 2010. – kennytm May 27, 2015 at 4:58

perhaps the `i >= 0` can be substituted for `i`. – Conor O'Brien Jun 27, 2015 at 2:35



51



If you use the library **underscore** you can write like this

```
var unionArr = _.union([34,35,45,48,49], [48,55]);
console.log(unionArr);
```

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/underscore.js/1.9.1/underscore-
min.js"></script>
```

Run code snippet

[Expand snippet](#)

Ref: <http://underscorejs.org/#union>

Share Improve this answer Follow

edited Feb 15, 2019 at 14:02



Tushar Walzade

3,737 ● 4 ● 33 ● 56

answered May 2, 2012 at 12:19



Codler

11k ● 6 ● 52 ● 65

- 2 I wouldn't normally reference a whole new library for one small piece of functionality, but the underscore library has a LOT of really useful functions to work with arrays. I also ended up using the `_intersection` and `_without` functions. – Ben Mills Feb 2, 2015 at 18:01

- 6 Lodash also has a [great implementation for union](#) – BGilman May 15, 2017 at 20:23



16



I'm probably wasting time on a dead thread here. I just had to implement this and went looking to see if I was wasting my time.

I really like KennyTM's answer. That's just how I would attack the problem. Merge the keys into a hash to naturally eliminate duplicates and then extract the keys. If you actually have jQuery you can leverage its goodies to make this a 2 line problem and then roll it into an extension. The `each()` in jQuery will take care of not iterating over items where `hasOwnProperty()` is false.

```
jQuery.fn.extend({
  union: function(array1, array2) {
    var hash = {}, union = [];
```

```
$.each($.merge($.merge([], array1), array2), function (index, value) {
  hash[value] = value; });
$.each(hash, function (key, value) { union.push(key); } );
return union;
});
```

Note that both of the original arrays are left intact. Then you call it like this:

```
var union = $.union(array1, array2);
```

Share Improve this answer Follow

answered Feb 2, 2011 at 20:35



Gareth Farrington

1,661 ● 1 ● 18 ● 24

I think this is the best answer there is because this function appears to work in $O(2n)$ (faster due to the two non-nested for loops) as opposed to the rest which appear to work in a $O(n^2)$ (slower) fashion.
– [Hengjie](#) Mar 2, 2013 at 0:31

Hengjie, Really $O(2n)$? I think, \$.merge has $O(n)$ or even $O(n*n)$ not $O(1)$. This can be optimized to $O(3n)$ if fill hash separately without merges – [vp_arth](#) Sep 23, 2013 at 5:47

1 @Hengjie: Kenny's answer is $O(3n)$ – [kumarharsh](#) Jan 19, 2014 at 10:30

3 $O(n) == O(2n) == O(3n)$ – [woodardj](#) Sep 24, 2016 at 23:09



10



If you wants to concatenate two arrays without any duplicate value,Just try this

```
var a=[34, 35, 45, 48, 49];
var b=[48, 55];
var c=a.concat(b).sort();
var res=c.filter((value,pos) => {return c.indexOf(value) == pos;} );
```

Share Improve this answer Follow

answered Jul 21, 2017 at 13:10



Narendra Manam

Narer 161 ● 3 ● 11

Great idea, but it change the order's items. – [Malus Jan](#) Sep 1, 2017 at 14:31

1 just remove sort method, It will solve your problem – [Narendra Manam](#) Oct 31, 2017 at 13:52

1 nice! or a variation if you are already in a method chaining (you don't have a reference to c): ``[1,2,3].concat([2,3,4]).filter((value, pos, arr)=>arr.indexOf(value)===pos)`` – [cancerbero](#) May 29, 2018 at 5:25



9

```
function unique(arrayName)
{
    var newArray=new Array();
    label: for(var i=0; i<arrayName.length;i++ )
    {
        for(var j=0; j<newArray.length;j++ )
        {
            if(newArray[j]==arrayName[i])
                continue label;
        }
        newArray[newArray.length] = arrayName[i];
    }
    return newArray;
}

var arr1 = new Array(0,2,4,4,4,4,5,5,6,6,6,7,8,9,5,1,2,3,0);
var arr2= new Array(3,5,8,1,2,32,1,2,1,2,4,7,8,9,1,2,1,2,3,4,5);
var union = unique(arr1.concat(arr2));
console.log(union);
```




Share Improve this answer Follow

edited Feb 15, 2019 at 13:36



Tushar Walzade

3,737 ● 4 ● 33 ● 56

answered Sep 2, 2010 at 18:01



Bob

3,074 ● 11 ● 43 ● 62

Maybe I'm just confused, but how would this work? You're passing arr3, but there is no arr3, and even if there was it doesn't look like it's actually a union on arr1 | arr2 like he requested? – Angelo R. Sep 2, 2010 at 18:06

2 I'm confused how this would not work? – Bob Sep 2, 2010 at 18:07

1 @Angelo R.: you're right: "there is no arr3." – BoltClock Sep 2, 2010 at 18:10

var arr2 = []; if (currTaskIDs != '') { if(\$.inArray(currTaskIDs, arr2) == -1) { arr2.push(currTaskIDs); } arr2 = unique(arr2.concat(arr)); } } now, currTaskIDs is = 34,35,45,48,49 if i select say 50, it removes all of them and just displays 50 – CFNinja Sep 2, 2010 at 18:11

by the way, if i just use this: var currTaskIDs = \$("#taskIDList").val(); // begin: create the task list: if (currTaskIDs != '') { if(\$.inArray(currTaskIDs, arr) == -1) { arr.push(currTaskIDs); } arr = unique(arr); } it works perfectly first time. if i add another value to my currTaskIDs, then it becomes something like : 34,35,45,48,49,50,34,35,45,48,49,50,38 – CFNinja Sep 2, 2010 at 18:14

Adapted from: <https://stackoverflow.com/a/4026828/1830259>

7

```
Array.prototype.union = function(a)
{
    var r = this.slice(0);
    a.forEach(function(i) { if (r.indexOf(i) < 0) r.push(i); });
    return r;
}
```



```

};

Array.prototype.diff = function(a)
{
    return this.filter(function(i) {return a.indexOf(i) < 0;});
};

var s1 = [1, 2, 3, 4];
var s2 = [3, 4, 5, 6];

console.log("s1: " + s1);
console.log("s2: " + s2);
console.log("s1.union(s2): " + s1.union(s2));
console.log("s2.union(s1): " + s2.union(s1));
console.log("s1.diff(s2): " + s1.diff(s2));
console.log("s2.diff(s1): " + s2.diff(s1));

// Output:
// s1: 1,2,3,4
// s2: 3,4,5,6
// s1.union(s2): 1,2,3,4,5,6
// s2.union(s1): 3,4,5,6,1,2
// s1.diff(s2): 1,2
// s2.diff(s1): 5,6

```

Share Improve this answer Follow

edited May 23, 2017 at 12:18

Community Bot
1 ● 1

answered Mar 20, 2014 at 2:13

Adam Fox
3,438 ● 5 ● 23 ● 20

I like Peter Ajtai's concat-then-unique solution, but the code's not very clear. Here's a nicer alternative:

6



```

function unique(x) {
    return x.filter(function(elem, index) { return x.indexOf(elem) === index; });
};
function union(x, y) {
    return unique(x.concat(y));
};

```

Since `indexOf` returns the index of the *first* occurrence, we check this against the current element's index (the second parameter to the filter predicate).

Share Improve this answer Follow

edited Sep 24, 2014 at 10:31

answered Aug 11, 2014 at 15:52

Warbo
2,636 ● 1 ● 29 ● 23

Shorter version of kennytm's answer:

4

```

function unionArrays(a, b) {
    const cache = {};

```

```

a.forEach(item => cache[item] = item);
b.forEach(item => cache[item] = item);

return Object.keys(cache).map(key => cache[key]);
};

```

Share Improve this answer Follow

answered Nov 12, 2016 at 0:30

 **worsnup**
 174 ● 9

You can use a jQuery plugin: [jQuery Array Utilities](#)

For example the code below

```
$.union([1, 2, 2, 3], [2, 3, 4, 5, 5])
```

will return [1,2,3,4,5]

Share Improve this answer Follow

answered Apr 25, 2013 at 19:09

 **Kristian Abrahamsen**
 111 ● 1 ● 4

This union function also supports an arbitrary number of arrays, This package [jquery-array-utilities](#) can also be installed using [bower](#). I would like to add a disclaimer that I originally made the plugin :)

– Kristian Abrahamsen Nov 28, 2016 at 21:28

```

function unite(arr1, arr2, arr3) {
  newArr=arr1.concat(arr2).concat(arr3);

  a=newArr.filter(function(value){
    return !arr1.some(function(value2){
      return value == value2;
    });
  });

  console.log(arr1.concat(a));

  }//This is for Sorted union following the order :)

```

Share Improve this answer Follow

answered Jul 4, 2015 at 10:50

 **Ademola Adegboyi**
 934 ● 1 ● 16 ● 25

2



```
function unionArrays() {
  var args = arguments,
      l = args.length,
      obj = {},
      res = [],
      i, j, k;

  while (l--) {
    k = args[l];
    i = k.length;

    while (i--) {
      j = k[i];
      if (!obj[j]) {
        obj[j] = 1;
        res.push(j);
      }
    }
  }

  return res;
}

var unionArr = unionArrays([34, 35, 45, 48, 49], [44, 55]);
console.log(unionArr);
```

[Run code snippet](#)[Expand snippet](#)

Somewhat similar in approach to alejandro's method, but a little shorter and should work with any number of arrays.

Share Improve this answer Follow

edited Feb 15, 2019 at 13:47



Tushar Walzade

3,737 ● 4 ● 33 ● 56

answered Feb 11, 2013 at 21:57



TimTucker

21 ● 1



2



```
function unionArray(arrayA, arrayB) {
  var obj = {},
      i = arrayA.length,
      j = arrayB.length,
      newArray = [];
  while (i--) {
    if (!(arrayA[i] in obj)) {
      obj[arrayA[i]] = true;
      newArray.push(arrayA[i]);
    }
  }
  while (j--) {
    if (!(arrayB[j] in obj)) {
      obj[arrayB[j]] = true;
      newArray.push(arrayB[j]);
    }
  }
}
```

```

    return newArray;
  }
  var unionArr = unionArray([34, 35, 45, 48, 49], [44, 55]);
  console.log(unionArr);

```

[Run code snippet](#)
[Expand snippet](#)

Faster <http://jsperf.com/union-array-faster>

Share Improve this answer Follow

edited Feb 15, 2019 at 13:54



Tushar Walzade

3,737 ● 4 ● 33 ● 56

answered Oct 12, 2012 at 0:40



alejandro

2,799 ● 1 ● 17 ● 25



2



I would first concatenate the arrays, then I would return only the unique value.

You have to create your own function to return unique values. Since it is a useful function, you might as well [add it in as a functionality of the Array](#).

In your case with arrays `array1` and `array2` it would look like this:

1. `array1.concat(array2)` - concatenate the two arrays
2. `array1.concat(array2).unique()` - return only the unique values. Here `unique()` is a method you added to the prototype for `Array`.

The whole thing would look like this:

```

Array.prototype.unique = function () {
  var r = new Array();
  o: for(var i = 0, n = this.length; i < n; i++)
  {
    for(var x = 0, y = r.length; x < y; x++)
    {
      if(r[x]==this[i])
      {
        continue o;
      }
    }
    r[r.length] = this[i];
  }
  return r;
}
var array1 = [34,35,45,48,49];
var array2 = [34,35,45,48,49,55];

// concatenate the arrays then return only the unique values
console.log(array1.concat(array2).unique());

```